Enterprise Disaster Recovery & Backup

Architecture

Project Overview

This project demonstrates enterprise-grade AWS storage and backup strategies including:

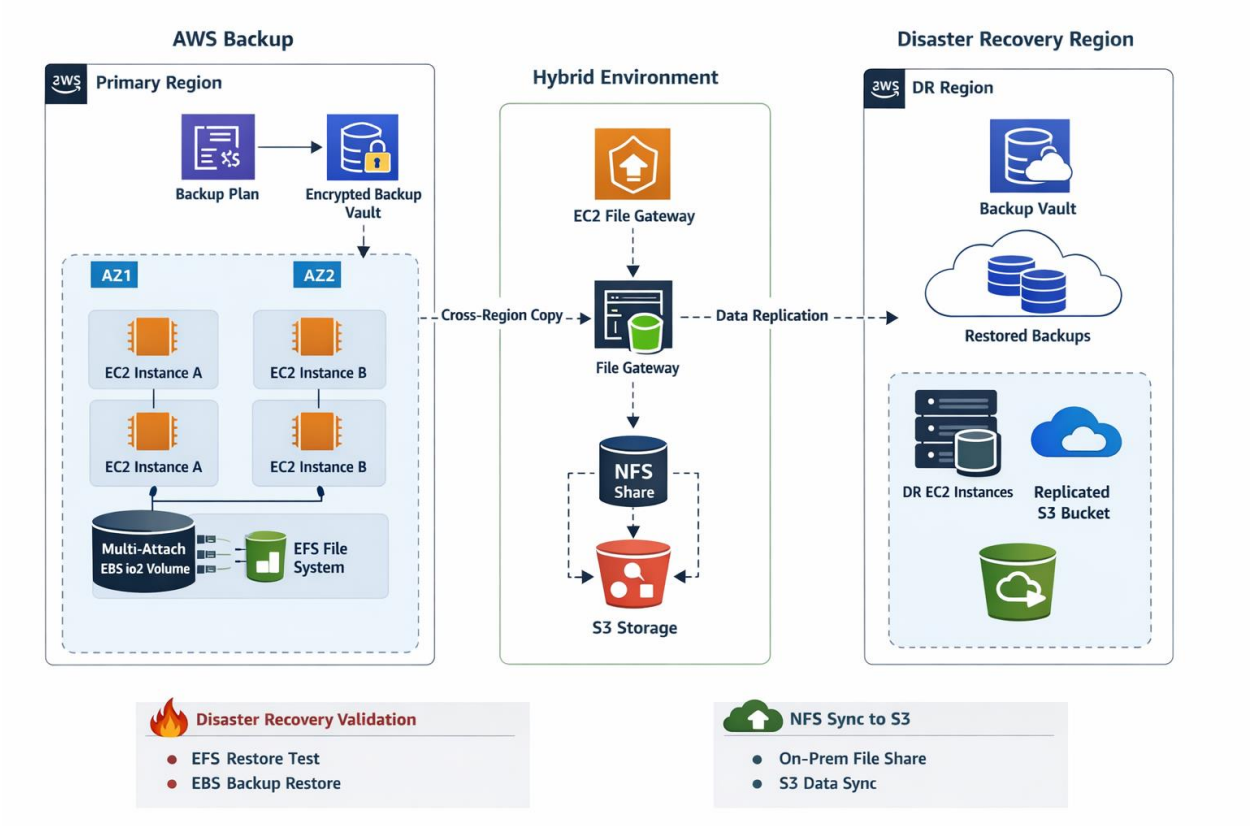
- AWS Backup with Vault & lifecycle policies
- Advanced EBS features (Multi-Attach + live resize)
- EFS shared storage with lifecycle management
- Hybrid Storage using File Gateway (simulated on EC2)

It also includes **disaster recovery validation scenarios** aligned with real-world **RTO/RPO goals**.

Architecture Diagram

Components:

- EC2 Instances (Multi-AZ)
- EBS io2 Multi-Attach Volume
- EFS File System
- AWS Backup Vault (KMS Encrypted)
- S3 Bucket (Storage Gateway Backend)
- File Gateway (on EC2)
- Cross-Region Backup (DR Region)



Services Used

- AWS Backup
- Amazon EBS
- Amazon EFS
- AWS KMS
- AWS Storage Gateway
- Amazon S3
- Amazon EC2

TASK 1 — AWS Backup Configuration

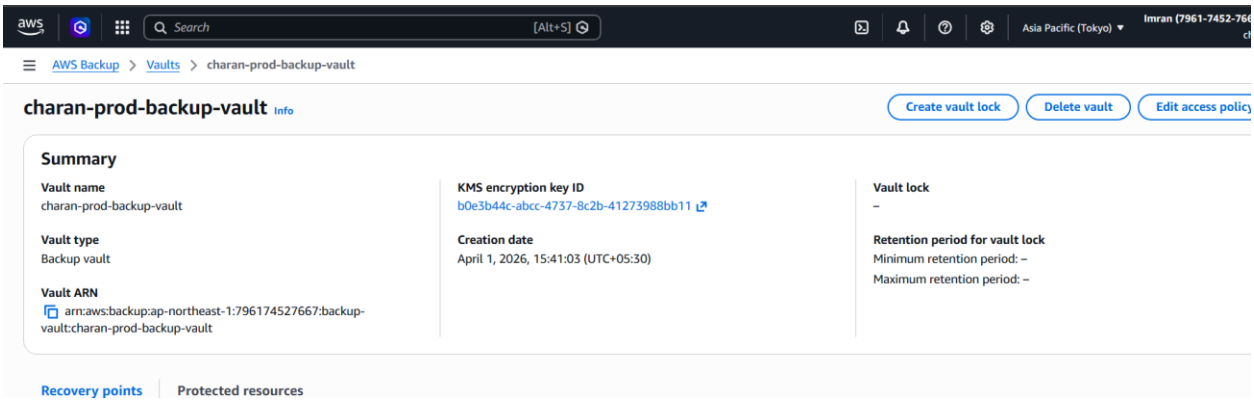
Objective

Configure centralized backup with encryption and lifecycle policies.

Steps

1. Create Backup Vault

- Go to **AWS Backup → Backup Vaults**
- Create vault:
  - Name: `charan-prod-backup-vault`
  - Encryption: Use **KMS Key**

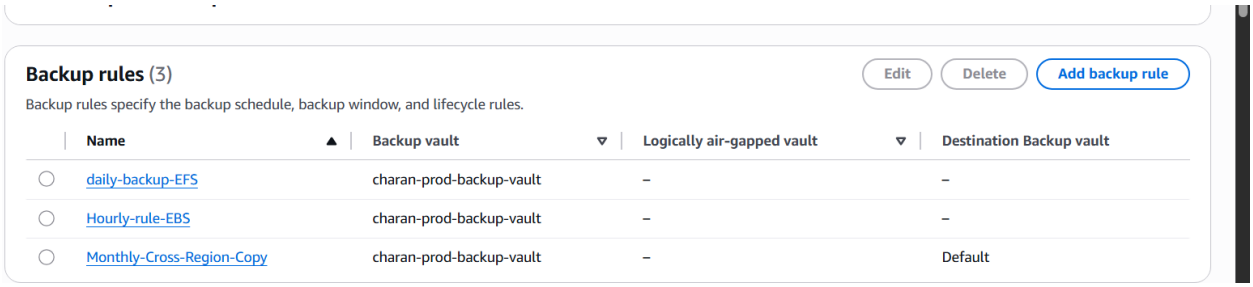


2. Create Backup Plan

- Go to **Backup Plans → Create Plan**

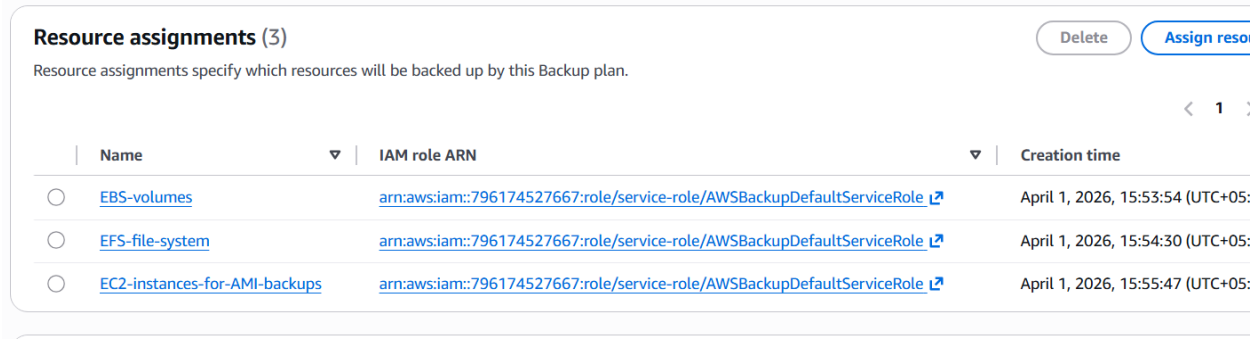
Plan Configuration:

- **Hourly Backup (EBS)**
  - Frequency: Every 1 hour
  - Retention: 24 hours
- **Daily Backup (EFS + AMI)**
  - Frequency: Daily
  - Retention: 30 days
- **Monthly Cross-Region Copy**
  - Destination Region: e.g., `eu-west-1`
  - Retention: 1 year



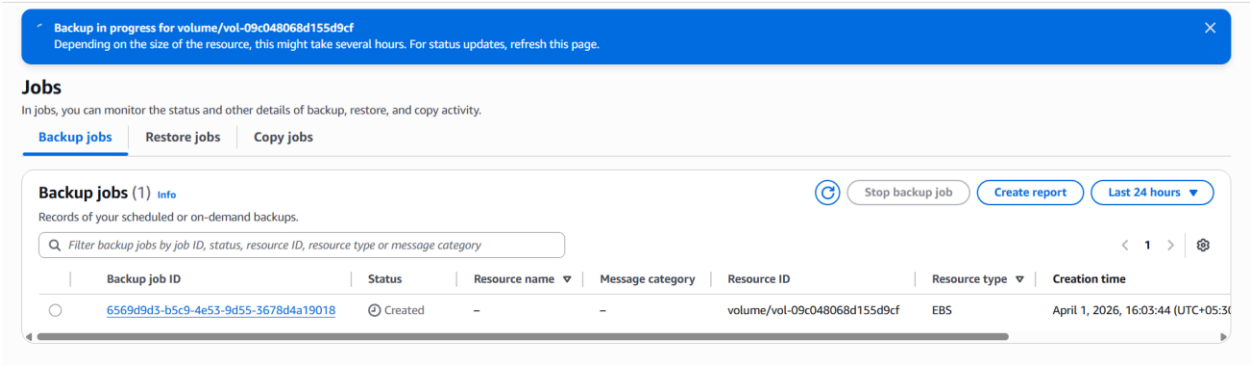
3. Assign Resources

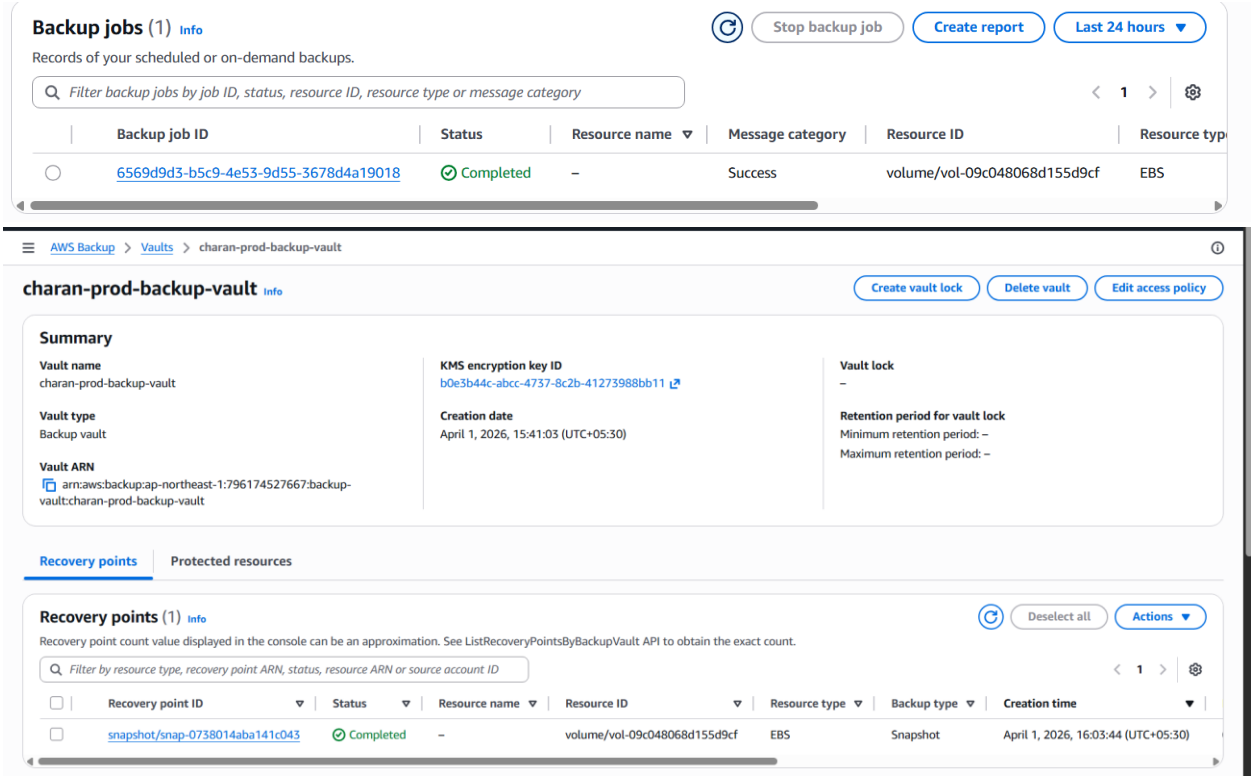
- Assign:
  - EBS volumes
  - EFS file system
  - EC2 instances (for AMI backups)



Validation

- Trigger **manual backup**
- Verify job success





TASK 2 — EBS Advanced Features

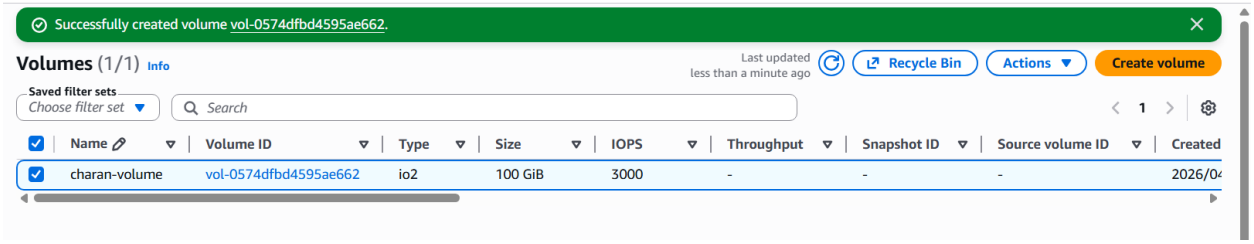
Objective

Demonstrate high-performance storage with zero downtime scaling.

Steps

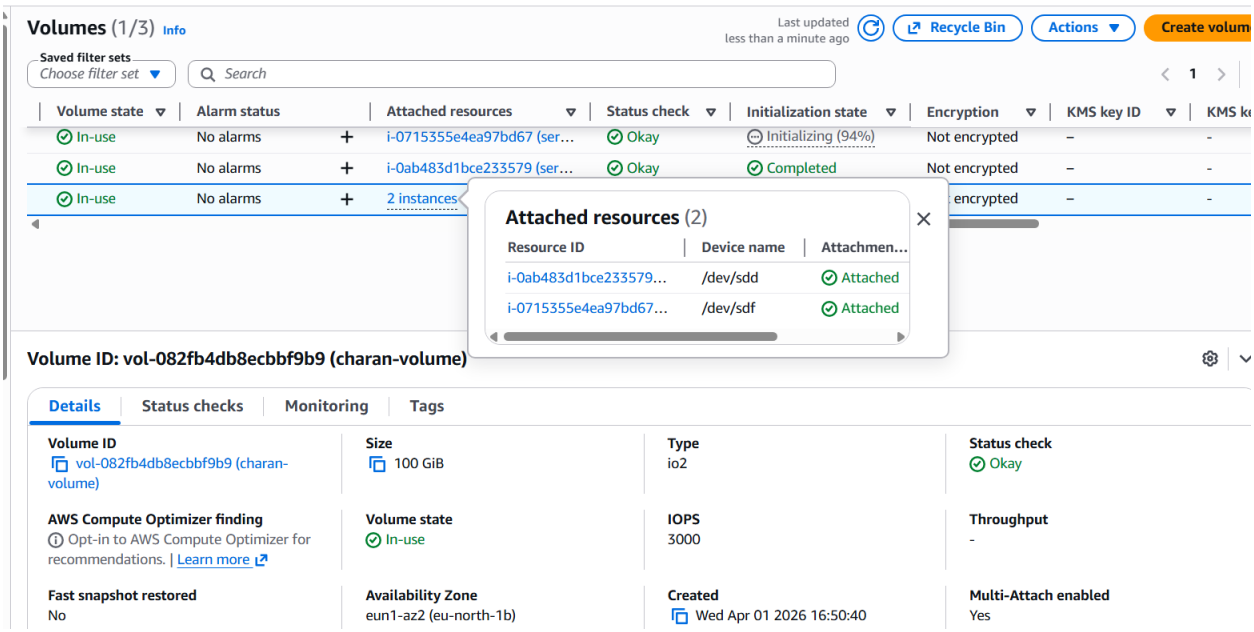
1. Create io2 Volume

- Size: 100 GB
- Type: io2
- Enable: **Multi-Attach**



2. Attach to 2 EC2 Instances

- Attach same volume to both instances



3. Mount Volume

```
lsblk
sudo mkfs -t ext4 /dev/nvme1n1
sudo mount /dev/nvme1n1/data
```

```
ubuntu@ip-172-31-35-97:~$ lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0          7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1          7:1    0   74M  1 loop /snap/core22/2339
loop2          7:2    0 48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    8G  0 disk
├─nvme0n1p1  259:1    0    7G  0 part /
├─nvme0n1p14 259:2    0    4M  0 part
├─nvme0n1p15 259:3    0  106M  0 part /boot/efi
└─nvme0n1p16 259:4    0  913M  0 part /boot
nvme1n1      259:5    0  100G  0 disk
ubuntu@ip-172-31-35-97:~$
```

```
ubuntu@ip-172-31-46-156:~$ lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0          7:0    0 27.8M  1 loop /snap/amazon-ssm-agent/12322
loop1          7:1    0   74M  1 loop /snap/core22/2339
loop2          7:2    0 48.1M  1 loop /snap/snapd/25935
nvme0n1      259:0    0    8G  0 disk
├─nvme0n1p1  259:1    0    7G  0 part /
├─nvme0n1p14 259:2    0    4M  0 part
├─nvme0n1p15 259:3    0  106M  0 part /boot/efi
└─nvme0n1p16 259:4    0  913M  0 part /boot
nvme1n1      259:5    0  100G  0 disk
ubuntu@ip-172-31-46-156:~$
```

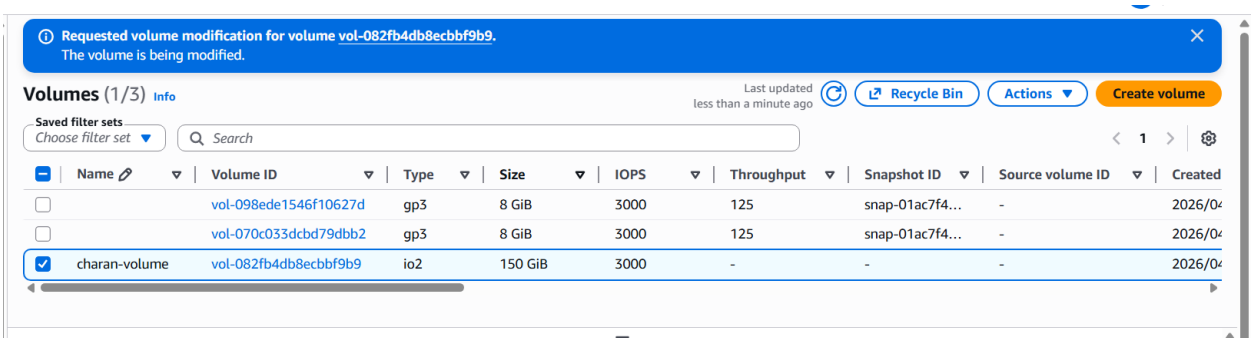
```
ubuntu@ip-172-31-46-156:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 26214400 4k blocks and 6553600 inodes
Filesystem UUID: 2ed85845-57c8-41f6-af50-e6b38c7c7f24
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000, 7962624, 11239424, 20480000, 23887872

Allocating group tables: done
Writing inode tables: done
Creating journal (131072 blocks): done
Writing superblocks and filesystem accounting information: done

ubuntu@ip-172-31-46-156:~$
```

```
ubuntu@ip-172-31-35-97:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  2.1G  4.7G  31% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  896K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1      98G   24K   93G   1% /home/ubuntu/data
ubuntu@ip-172-31-35-97:~$
```

4. Resize Volume (100GB → 150GB)

Step 1: Modify Volume

- Increase size to 150GB in console



Step 2: Extend Filesystem

```
sudo growpart /dev/nvme1n1
sudo resize2fs /dev/nvme1n1
```

```
ubuntu@ip-172-31-35-97:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  2.1G  4.7G  31% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  896K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1     148G   24K  141G   1% /home/ubuntu/data
ubuntu@ip-172-31-35-97:~$
```

Validation

- Verify both instances can read/write
- Confirm new disk size

```
df -h
```

```
ubuntu@ip-172-31-46-156:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.8G  2.1G  4.7G  31% /
tmpfs            456M     0  456M   0% /dev/shm
tmpfs            183M  896K  182M   1% /run
tmpfs            5.0M     0  5.0M   0% /run/lock
efivarfs         128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   94M  726M  12% /boot
/dev/nvme0n1p15  105M  6.2M   99M   6% /boot/efi
tmpfs             92M   12K   92M   1% /run/user/1000
/dev/nvme1n1     148G   24K  141G   1% /home/ubuntu/data
ubuntu@ip-172-31-46-156:~$
```

TASK 3 — EFS Setup

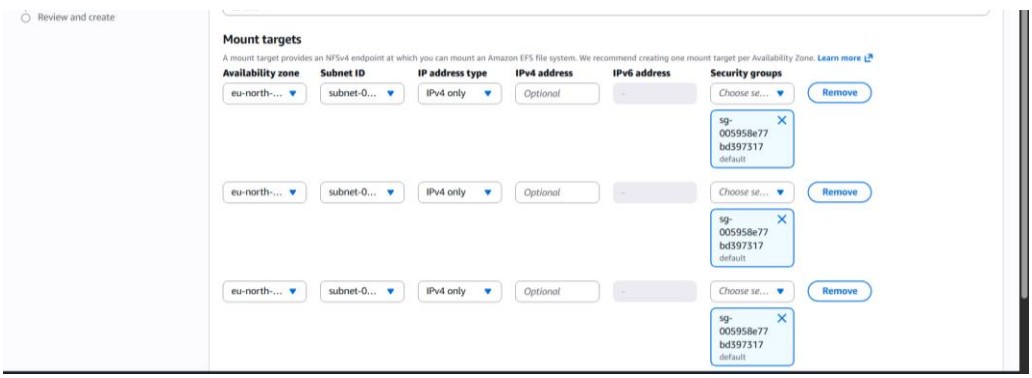
Objective

Set up shared file storage across multiple AZs with lifecycle policies.

Steps

1. Create EFS File System

- Enable across **2 AZs**
- Performance mode: General Purpose



Step 1: File system settings

File system

Field	Value
Name	charan-efs
Performance mode	General Purpose
Throughput mode	Elastic
Encrypted	Yes
KMS Key ID	-
	Transition into Infrequent Access (IA): 30 day(s

2. Configure Lifecycle Policy

- IA transition: 30 days
- Archive transition: 90 days

Lifecycle management

Automatically save money as access patterns change by moving files into the Infrequent Access (IA) or Archive storage class. Learn more

Transition into Infrequent Access (IA)
Transition files to IA based on the time since they were last accessed in Standard storage.
30 day(s) since last access

Transition into Archive
Transition files to Archive based on the time since they were last accessed in Standard storage.
90 day(s) since last access

Transition into Standard
Transition files back to Standard storage based on when they are first accessed in IA or Archive storage.
None

3. Mount on EC2 Instances (3 Instances)

```
sudo apt install nfs-common -y
sudo mount -t nfs4 <efs-endpoint>:/ /mnt/efs
```

```
ubuntu@ip-10-0-2-241:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvpor
t fs-0b4dcf32a9290d165.efs.ap-northeast-1.amazonaws.com:/ efs-1
ubuntu@ip-10-0-2-241:~$ df -h
Filesystem                                          Size  Used Avail Use% Mounted on
/dev/root                                           6.8G  2.1G  4.7G  31% /
tmpfs                                               456M     0  456M   0% /dev/shm
tmpfs                                               183M  904K  182M   1% /run
tmpfs                                               5.0M     0  5.0M   0% /run/lock
efivarfs                                            128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16                                     881M   94M  726M  12% /boot
/dev/nvme0n1p15                                     105M  6.2M   99M   6% /boot/efi
tmpfs                                                92M   12K   92M   1% /run/user/1000
fs-0b4dcf32a9290d165.efs.ap-northeast-1.amazonaws.com:/  8.0E     0  8.0E   0% /home/ubuntu/efs-1
ubuntu@ip-10-0-2-241:~$
```

```
ubuntu@ip-10-0-4-195:~$ df -h
Filesystem                                          Size  Used Avail Use% Mounted on
/dev/root                                           6.8G  2.1G  4.7G  31% /
tmpfs                                               456M     0  456M   0% /dev/shm
tmpfs                                               183M  904K  182M   1% /run
tmpfs                                               5.0M     0  5.0M   0% /run/lock
efivarfs                                            128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16                                     881M   94M  726M  12% /boot
/dev/nvme0n1p15                                     105M  6.2M   99M   6% /boot/efi
tmpfs                                                92M   12K   92M   1% /run/user/1000
fs-0b4dcf32a9290d165.efs.ap-northeast-1.amazonaws.com:/  8.0E     0  8.0E   0% /home/ubuntu/efs-2
ubuntu@ip-10-0-4-195:~$
```

```
ubuntu@ip-10-0-6-52:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport
 fs-0b4dcf32a9290d165.efs.ap-northeast-1.amazonaws.com:/ efs-3
ubuntu@ip-10-0-6-52:~$ df -h
Filesystem                                          Size  Used Avail Use% Mounted on
/dev/root                                           6.8G  2.1G  4.7G  31% /
tmpfs                                               456M     0  456M   0% /dev/shm
tmpfs                                               183M  904K  182M   1% /run
tmpfs                                               5.0M     0  5.0M   0% /run/lock
efivarfs                                            128K  3.6K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p16                                     881M   94M  726M  12% /boot
/dev/nvme0n1p15                                     105M  6.2M   99M   6% /boot/efi
tmpfs                                                92M   12K   92M   1% /run/user/1000
fs-0b4dcf32a9290d165.efs.ap-northeast-1.amazonaws.com:/  8.0E     0  8.0E   0% /home/ubuntu/efs-3
ubuntu@ip-10-0-6-52:~$
```

Validation

- Write file from one instance
- Verify visibility on others

echo "test" > /mnt/efs/file.txt
sudo chown ubuntu:ubuntu /home/ubuntu/efs (not to get permission denied)

```
ubuntu@ip-10-0-4-195:~$ sudo chown ubuntu:ubuntu /home/ubuntu/efs-2
ubuntu@ip-10-0-4-195:~$ echo "test" > /home/ubuntu/efs-2/file.txt
ubuntu@ip-10-0-4-195:~$ cd efs-2/
ubuntu@ip-10-0-4-195:~/efs-2$ ls
file.txt
ubuntu@ip-10-0-4-195:~/efs-2$
```

```
ubuntu@ip-10-0-2-241:~/efs-1$ ls
file.txt
ubuntu@ip-10-0-2-241:~/efs-1$
```

```
ubuntu@ip-10-0-6-52:~$ cd efs-3/
ubuntu@ip-10-0-6-52:~/efs-3$ ls
file.txt
ubuntu@ip-10-0-6-52:~/efs-3$
```

Storage Gateway (Hybrid Setup)

Objective

Simulate on-prem to cloud file sync using File Gateway.

Steps

1. Launch EC2 for File Gateway

- Instance Type: m5.large (recommended) with 80gb as root but 30 gb can be used to run the gateway.
- Attach additional disk (150 gb) for caching.

2. Install File Gateway

- Deploy AWS Storage Gateway
- Activate gateway

3. Configure File Share (NFS)

- Backend: S3 Bucket
- Enable: Cross-region replication for DR

4. Mount NFS Share

```
sudo mount -t nfs <gateway-ip>:/share /mnt/gateway
```

5. Write Data

```
echo "Hello Hybrid Cloud" > /mnt/gateway/test.txt
```

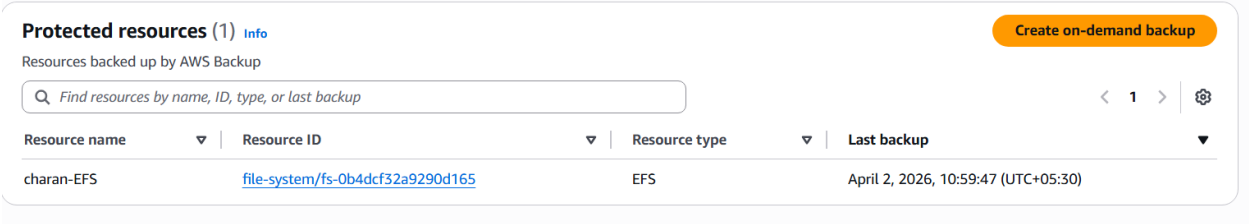
Validation

- Verify file in S3 bucket
- Confirm replication in DR region

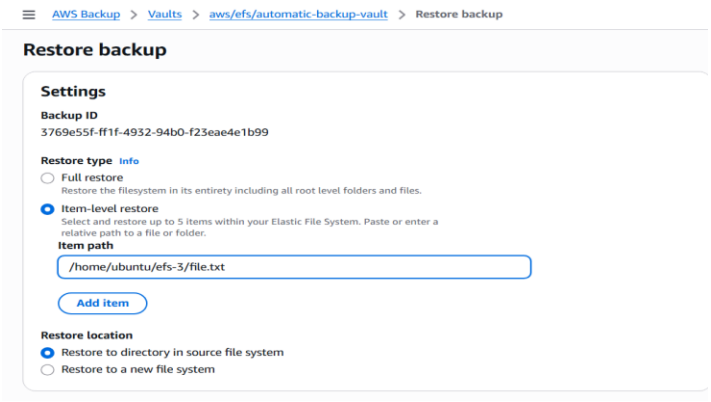
Disaster Recovery Testing

Scenario 1 — EFS File Deletion

- Delete file from EFS
- Restore using AWS Backup



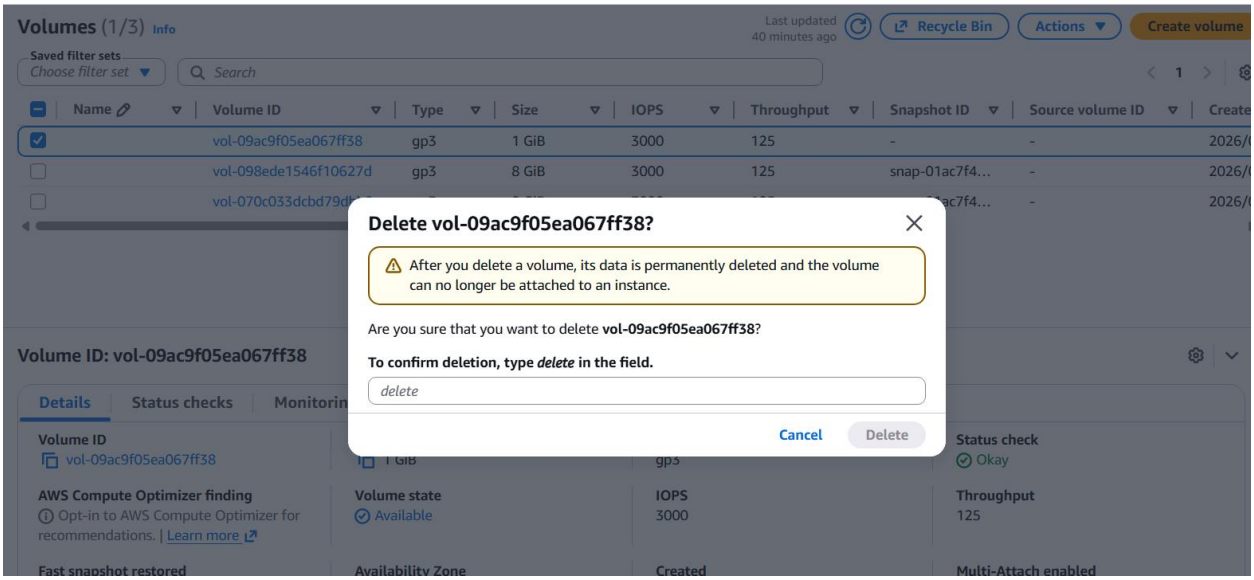

```
ubuntu@ip-10-0-6-52:~/efs-3$ rm file.txt
ubuntu@ip-10-0-6-52:~/efs-3$ ls
ubuntu@ip-10-0-6-52:~/efs-3$
```

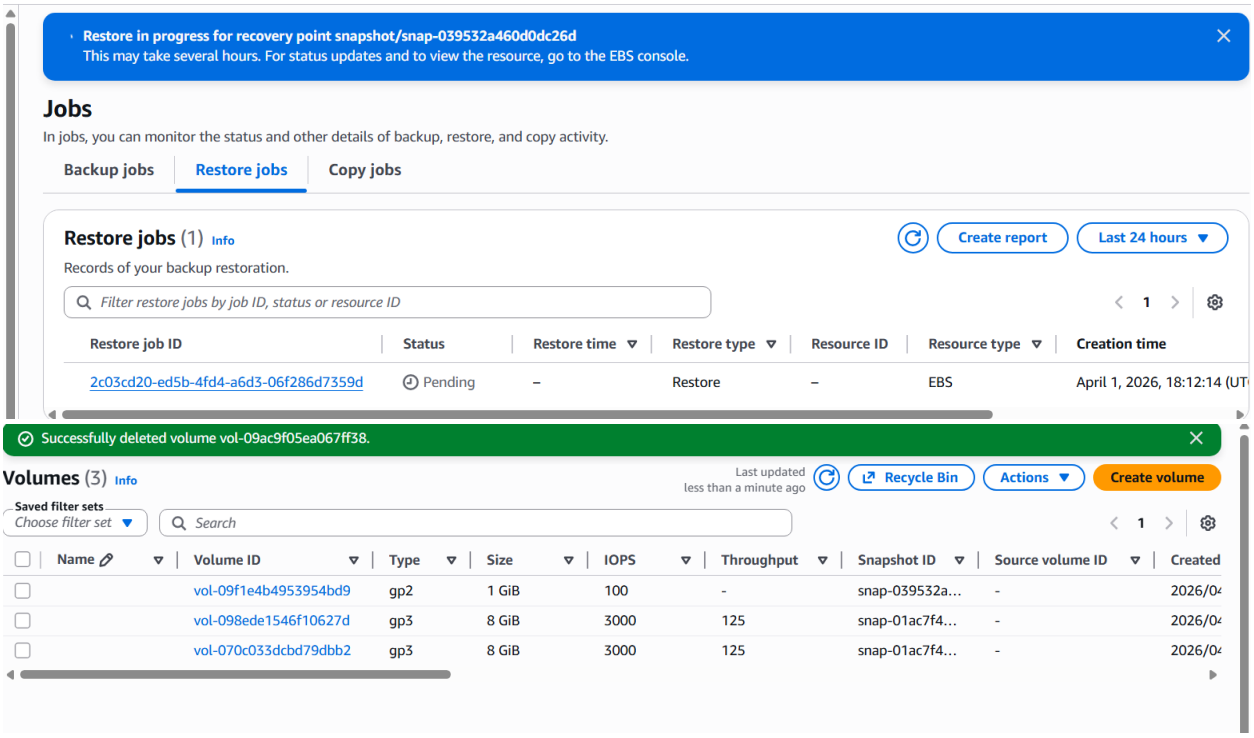



```
ubuntu@ip-10-0-6-52:~/efs-3$ ls
file.txt
ubuntu@ip-10-0-6-52:~/efs-3$
```

Scenario 2 — Backup Restore

- Restore EBS / EFS from backup
- Verify data integrity





Key Concepts Covered

- AWS Backup (Vaults, Plans, Lifecycle)
- KMS Encryption
- EBS Multi-Attach
- Live Volume Resize
- EFS Lifecycle Management

- Hybrid Storage (File Gateway)
- Disaster Recovery (RTO/RPO)

Validation Checklist

✓ Manual backup triggered and completed

✓ EFS restore successful after deletion

✓ Multi-Attach works across 2 instances

✓ EBS resized without downtime

✓ Storage Gateway syncs files to S3

✓ Cross-region replication verified